

## 3.4 — Determine When to Use Plan Mode vs Direct Execution

### Concept

Claude Code offers two primary execution modes: **Plan Mode** and **Direct Execution**. Choosing the right mode is a judgment call that balances thoroughness against speed. Using the wrong mode in either direction has real costs — planning too much slows down simple tasks, while diving directly into complex changes increases the risk of cascading mistakes.

#### Plan Mode (Shift+Tab)

Plan Mode activates a structured think-first workflow. Claude reads and investigates the codebase without making changes, produces a written implementation plan, and waits for your approval before executing. This mode is appropriate when the cost of a wrong move is high.

##### Trigger Plan Mode when:

- The task touches **3 or more files**, especially across different modules or layers
- The change involves **cross-cutting concerns** — authentication, error handling, logging, database schema migrations, API contracts
- The **risk level is HIGH**: production systems, shared libraries, public APIs, or anything that affects other developers' work
- You are making an **architectural decision** — choosing a pattern, restructuring a directory, introducing a new abstraction
- The task description is **ambiguous** and proceeding without clarification would likely require rework
- You are working in an **unfamiliar codebase** and need to understand the lay of the land before touching anything

#### Direct Execution

Direct execution skips the planning phase and immediately applies changes. It is correct for narrow, well-understood work where the scope is clear and the blast radius is small.

##### Use Direct Execution when:

- The change is **isolated to a single file** with no downstream effects
- The task is **mechanical and low-risk**: renaming a variable, fixing a typo, updating a constant, adding a comment
- You already have a plan and are executing a **specific, pre-defined step**
- The **planning overhead would exceed the implementation effort** — for a two-line fix, a five-minute planning session is waste

### The Explore Subagent

The **Explore subagent** is a read-only investigation mode. It allows Claude to traverse the codebase, read files, and build understanding without modifying anything. Use it as a precursor to Plan Mode when you need codebase context before committing to a plan.

Explore is particularly valuable for:

- Mapping dependency graphs before a refactor
- Identifying all call sites of a function before changing its signature
- Understanding data flow across layers before modifying a shared model
- Auditing configuration or environment variable usage before adding new infrastructure

The pattern is: **Explore first → Plan → Execute**. Skipping Explore on an unfamiliar codebase before a large change is a common source of incomplete plans.

### Complexity Assessment Criteria

Use this rubric to decide which mode to apply:

| Factor                    | Low Complexity (Direct)    | High Complexity (Plan)               |
|---------------------------|----------------------------|--------------------------------------|
| Files affected            | 1                          | 3+                                   |
| Cross-cutting concerns    | None                       | Yes (auth, logging, schema)          |
| Risk level                | Dev/test only              | Production / shared API              |
| Familiarity with codebase | High                       | Low                                  |
| Reversibility             | Easy (local, no migration) | Hard (DB migration, contract change) |
| Ambiguity in requirements | None                       | Present                              |

No single factor forces one mode. Evaluate the aggregate — a two-file change that touches authentication schema is still a Plan Mode task.

### Code Example

```
# Scenario A: Direct Execution is correct
Task: "Change the default timeout constant from 30 to 60 seconds in config.ts"

Assessment:
- 1 file affected
- No cross-cutting concerns
- Trivially reversible
→ Direct Execution

# Scenario B: Plan Mode is correct
Task: "Add rate limiting to all authenticated API endpoints"

Assessment:
- Affects route handlers, middleware stack, and potentially the auth layer
- Cross-cutting concern: all authenticated routes
```

- Risk: production endpoints, affects all users
- Plan Mode + Explore subagent to map all route files first

# Scenario C: Explore then Plan

Task: "Refactor the UserRepository to use a new ORM"

Assessment:

- Scope is unknown – how many files call UserRepository?
- Schema and query logic will change
- Run Explore subagent to identify all consumers
- Plan Mode to define migration steps
- Direct Execution per step with review gates

## Common Mistakes

---

**Over-planning simple changes.** Triggering Plan Mode for a single-file, low-risk edit adds friction without reducing risk. Direct execution is the right default for clearly bounded tasks.

**Under-planning architectural changes.** The most costly mistake is directly executing a refactor that touches shared interfaces or cross-cutting concerns without a plan. Partial changes that break the build mid-way are harder to recover from than taking five minutes to plan upfront.

**Skipping Explore before planning in unfamiliar code.** A plan built on incomplete knowledge of the codebase will miss call sites, hidden dependencies, or conflicting patterns. Explore is not optional when working in code you did not write.

**Conflating plan approval with implementation correctness.** Approving a plan is a gate on the approach, not a guarantee the execution will be correct. Continue reviewing individual edits even after approving a plan.

**Treating Plan Mode as only for large features.** Plan Mode is equally appropriate for any decision that sets a precedent — a new file structure, a new error handling convention, a new test pattern. Architectural decisions are high-leverage regardless of line count.

## References

---

- Claude Code documentation: Plan Mode (Shift+Tab)
- Claude Code documentation: Explore subagent
- Exam domain 3: Tool Use and Execution Strategy
- See also: 1.6 — Design Task Decomposition Strategies for Complex Workflows